

Student name: Gordon Tse

Student ID: s223532749

SIT225: Data Capture Technologies

Activity 8.1: Using smartphone to capture sensor data

The **Arduino IoT Remote** phone application lets you control and monitor all of your dashboards in the Arduino Cloud. With the app, you can also access your phone's internal sensors such as GPS data, light sensor, IMU and more (depending on what phone you have).

The phone's sensor data is automatically stored in Cloud variables, which you can also synchronize with other Things such as custom thing in Python board. This means your phone can become a part of your IoT system, acting as another node in your network.

In this activity, you will enable your smartphone to work as a custom device (like an Arduino board) and connect to your smartphone sensors such as accelerometers and GPS and streaming data to Arduino IoT Cloud dashboard.

Hardware Required

Your smartphone – compatible Android or iPhone

NOTE: *The IoT Remote app requires iOS 12.4 or later for iOS the version. If you are using Android, version 8.0 or later is required. Make sure the iOS or Android version on your device is up to date before downloading the app.*

Software Required

Android / iOS smart phone.

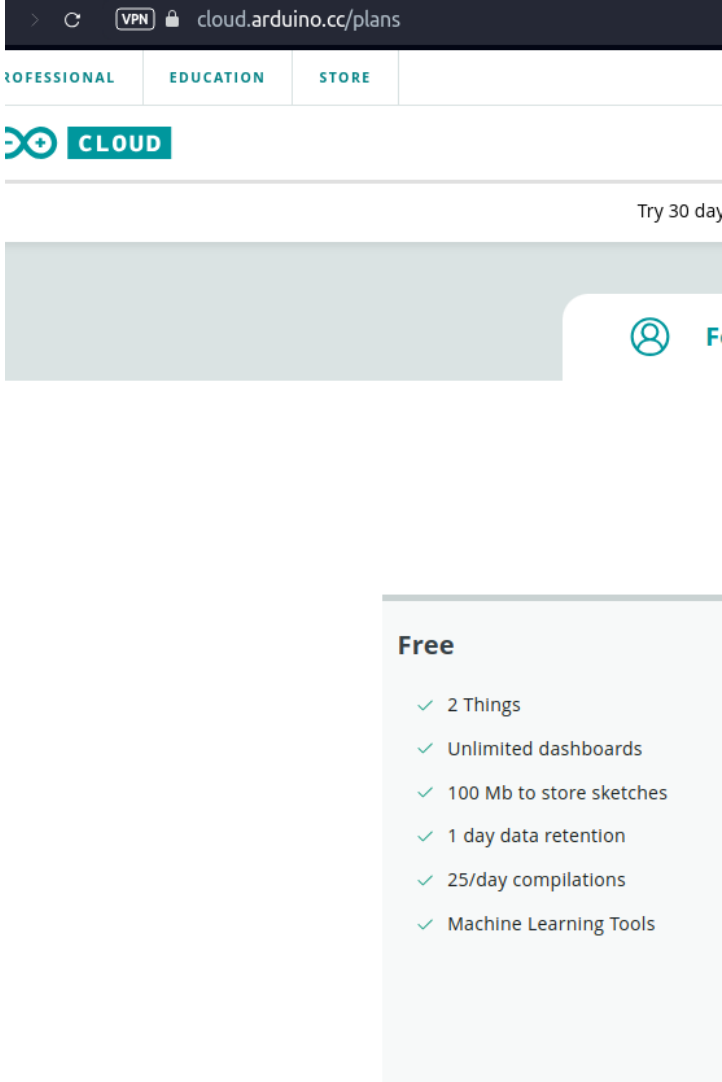
Arduino account

Arduino IoT Remote App (App Store or Google Play)

Python 3 (for custom Python Thing)

Steps

Step	Action
1	Install App: To use the Arduino IoT Remote app, visit Google Play / App Store and search for "Arduino IoT Remote". After installing the app, you will need to log in to your Arduino account. After you login, you will discover all your dashboards (if you have any), in the main menu. Based on the app version, home screen may vary. There will be 3 tabs at the bottom – Dashboards, Devices and Activity. You can follow the tutorial (https://docs.arduino.cc/arduino-cloud/iot-remote-app/getting-started).
2	Add device: Tap into the Devices tab. You will be able to create a new device. Alternatively, you can your profile (top right corner), in the settings section, you will see “Phone as device” which you can turn ON if it is OFF. There, you can select sensors in your smartphone such as accelerometer linear, accelerometer x/y/z and GPS among others. Note: A free account is enough for this experiment. If you are asked to upgrade your account, you can remove all other Things from your Arduino IoT Cloud account since the Free account allows at most 2 Things to configure, see below image.

	 The add device wizard will allow you to setup your sensor and also create a dashboard which you can see in your smartphone app. If you login to Arduino IoT Cloud in web browser, you can see the dashboard for your smartphone is already created.
3	Keep your smartphone screen ON for a while: Keep data coming through your smartphone for 10-15 minutes. During this time, keep moving your smartphone in a pattern so the accelerometer data can be analysed to discover the pattern. You can download data from the Arduino Cloud dashboard page by clicking on a download icon at top right corner which shows – Download historic data. A data download link will be sent to your account email from there you can download data.

	Question: Take a screenshot of your Ardiuno Cloud Dashboard where smartphone data is streaming and paste it here.
--	---

	Answer:
--	-----------------------

3:51

88



iPhone Dashboard



Accelerometer X

0.187

Accelerometer Y

-0.688

Accelerometer Z

-0.713

Linear accelerometer (m/s²)



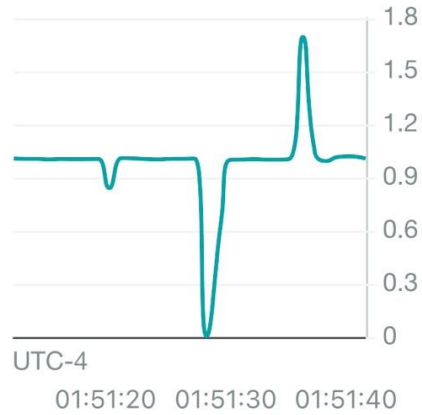
15D

7D

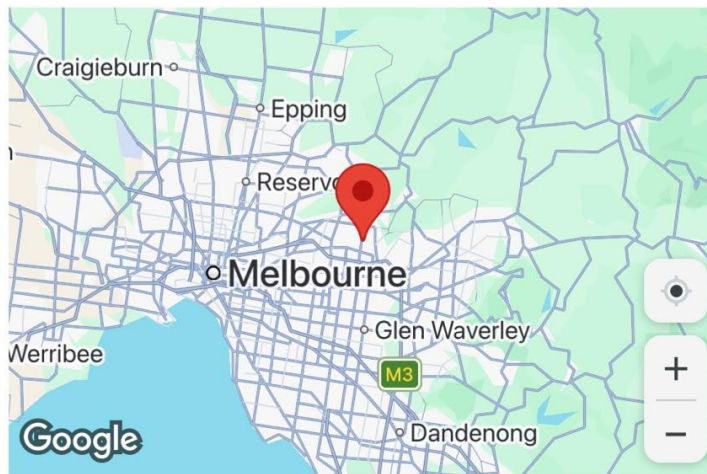
1D

1H

LIVE



GPS



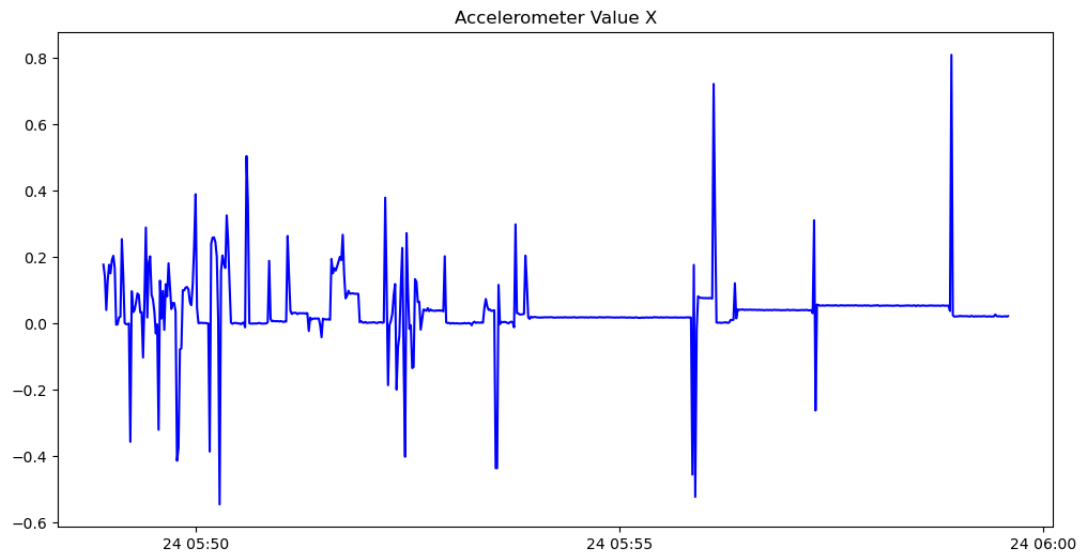
4

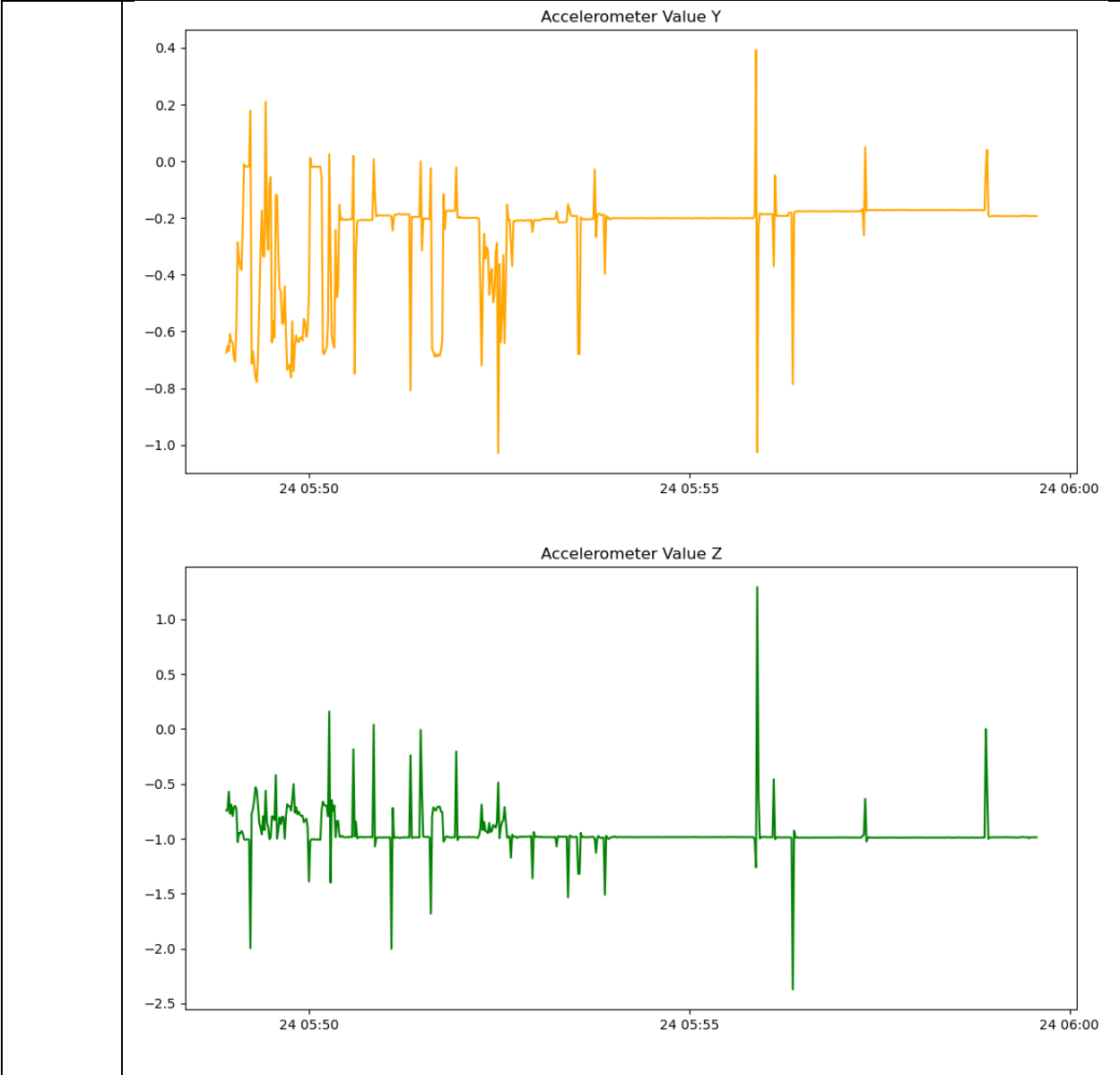
Plot accelerometer data:

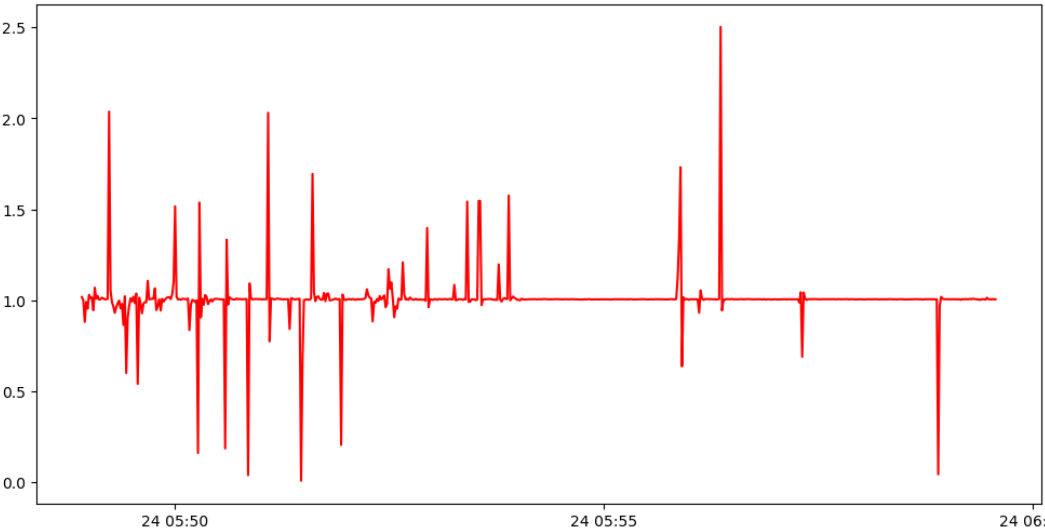
The zipped data file you downloaded from the cloud contains separate files per variable including accelerometer_linear, accelerometer_x, accelerometer_y, accelerometer_z and Gps. Each file has 2 columns – time and value.

Question: Open Jupyter Notebook by using command line, go to the data folder and write command (`$ jupyter lab`). Using Pandas, read CSV file and fetch the data column for accelerometer_x and plot it using Python plotting library (matplotlib or any other convenient for you). Repeat the plotting process for accelerometer y and z to have 3 separate graphs. Now create a fourth graph with all 3 variables x, y and z. Screenshot the 4 graphs and paste here.

Answer:





	Accelerometer Data Linear 
5	Question: Analyse accelerometer variables to find any repeating pattern. Remember that you were repeatedly moving your phone in a single pattern which should be manifested in the graphs. Justify your answer. Answer: I found out that when I move my phone all three graphs will have movement, combine the 3 x,y,z graph will become linear chart which is overall movement. When I stop touching my phone all 3 graphs become stable and it lead to linear chart also stable. They all have the same gap between next large movement because I move my phone in fix amount of time.

Activity 8.2: Receive smartphone sensor data from Python script

You can connect anything to Arduino Cloud including a wide range of compatible Arduino boards such as Arduino Nano 33 IoT or a third-party device that speaks Python. In activity 3.2, you have configured custom Python board and created a cloud variable that was synced to your Arduino Thing such as DHT22 sensor variables.

In this activity, you will need to synchronise smartphone's accelerometer x, y and z variables to Python script. If you can recall, you have already done a similar function in Activity 3.2.

Steps:

Step	Action
1	Configure Python board in Arduino Cloud and create a Thing where define 3 variables at a time and sync to corresponding accelerometer variable of smartphone Thing.
2	Write Python script to keep listening to data from the 3 variables to come through. You may need to create 3 call-back functions – a single function per variable (x, y and z).
3	Question: Keep storing each variable data in a separate file. Append each value with a timestamp so each data reading forms a comma separated line - <timestamp>, <data-value>. New data is written in a separate line. Keep storing them in a CSV file, where there will be 3 separate files. Screenshot your Python script here and screenshot the files opened side-by-side you have created and paste it here. Answer:

catch_phone_sensor.py

C:\> Users > gordo > OneDrive > 桌面 > Deakin > Y3 > T2 > SIT225 Data Capture > SIT225_2026T2 > Week_8 > catch

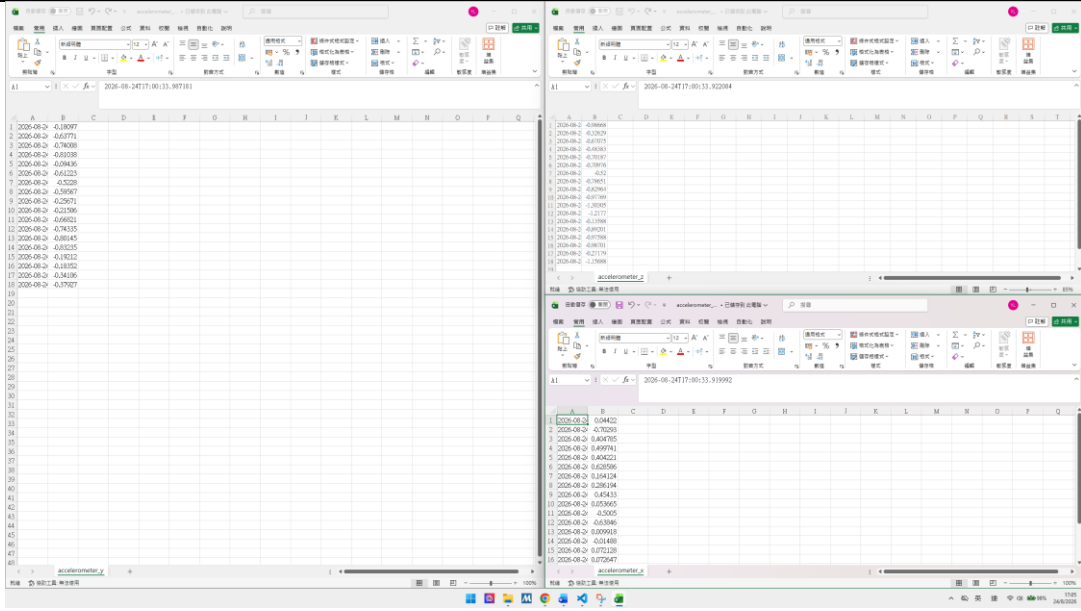
```
19
20 def on_axis_update(client, value, axis_name):
21     timestamp = datetime.now().isoformat()
22     latest_readings[axis_name] = value
23
24     # Write to individual file
25     file_path_indiv = os.path.join(
26         SCRIPT_DIR, f"accelerometer_{axis_name}.csv"
27     )
28     with open(file_path_indiv, "a", newline="") as f:
29         writer = csv.writer(f)
30         writer.writerow([timestamp, value])
31         f.flush()
32
33     # Step 4: Write to combined file
34     if all(val is not None for val in latest_readings.values()):
35         file_path_comb = os.path.join(SCRIPT_DIR, "accelerometer_combined.csv")
36         with open(file_path_comb, "a", newline="") as f:
37             writer = csv.writer(f)
38             writer.writerow(
39                 [
40                     timestamp,
41                     latest_readings["x"],
42                     latest_readings["y"],
43                     latest_readings["z"],
44                 ]
45             )
46             f.flush()
47
48     print(
49         f"Logged {axis_name.upper()}: {value} -> Saved to {file_path_indiv}"
50     )
51
52
53 # Register callbacks
54 client.register(
55     "python_x", on_write=lambda c, v: on_axis_update(c, v, "x")
56 )
57 client.register(
58     "python_y", on_write=lambda c, v: on_axis_update(c, v, "y")
59 )
```

01 701 0 0 0

4

Question: Now manage 3 variable data so they can be stored in a single CSV file where each line consists of comma separated sensor values with a timestamp - <timestamp>, <x>, <y>, <z>. Store data once you gather 3 variables and repeat the process. Screenshot your Python script here and screenshot the file you have created opened and paste it here.

Answer:



catch_phone_sensor.py

C: > Users > gordo > OneDrive > 桌面 > Deakin > Y3 > T2 > SIT225 Data Capture > SIT225_2026T2 > Week_8 > catch_p

```
20 def on_axis_update(client, value, axis_name):
21     | f.flush()
22
23     # Step 4: Write to combined file
24     if all(val is not None for val in latest_readings.values()):
25         file_path_comb = os.path.join(SCRIPT_DIR, "accelerometer_combined.csv")
26         with open(file_path_comb, "a", newline="") as f:
27             writer = csv.writer(f)
28             writer.writerow(
29                 | [
30                     | timestamp,
31                     | latest_readings["x"],
32                     | latest_readings["y"],
33                     | latest_readings["z"],
34                 | ]
35             )
36         f.flush()
37
38     print(
39         | f"Logged {axis_name.upper()}: {value} -> Saved to {file_path_indiv}"
40     )
41
42 # Register callbacks
43 client.register(
44     | "python_x", on_write=lambda c, v: on_axis_update(c, v, "x")
45 )
46 client.register(
47     | "python_y", on_write=lambda c, v: on_axis_update(c, v, "y")
48 )
49 client.register(
50     | "python_z", on_write=lambda c, v: on_axis_update(c, v, "z")
51 )
52
53 print("Listening for smartphone stream...")
54 client.start()
```

自動儲存 關閉 accelerometer_combined.csv

檔案 常用 插入 繪圖 頁面配置 公式 資料 校閱 檢視 自動化 說明

剪下 貼上 複製 複製格式 剪貼簿

新細明體 12 A⁺ A⁻ B I U 字型 對齊

A1 : X ✓ f_x

	A	B	C	D	E	F	G
1		0.04422	-0.18097	-0.98668			
2	2026-08-24	-0.70293	-0.18097	-0.98668			
3	2026-08-24	-0.70293	-0.63771	-0.98668			
4	2026-08-24	-0.70293	-0.63771	-0.32629			
5	2026-08-24	0.404785	-0.63771	-0.32629			
6	2026-08-24	0.404785	-0.74008	-0.32629			
7	2026-08-24	0.404785	-0.74008	-0.67075			
8	2026-08-24	0.499741	-0.74008	-0.67075			
9	2026-08-24	0.499741	-0.81038	-0.67075			
10	2026-08-24	0.499741	-0.81038	-0.48383			
11	2026-08-24	0.404221	-0.81038	-0.48383			
12	2026-08-24	0.404221	-0.09436	-0.48383			
13	2026-08-24	0.404221	-0.09436	-0.70187			
14	2026-08-24	0.628586	-0.09436	-0.70187			
15	2026-08-24	0.628586	-0.61223	-0.70187			
16	2026-08-24	0.628586	-0.61223	-0.70976			
17	2026-08-24	0.164124	-0.61223	-0.70976			
18	2026-08-24	0.164124	-0.5228	-0.70976			
19	2026-08-24	0.164124	-0.5228	-0.52			
20	2026-08-24	0.286194	-0.5228	-0.52			
21	2026-08-24	0.286194	-0.59567	-0.52			
22	2026-08-24	0.286194	-0.59567	-0.78651			
23	2026-08-24	0.45433	-0.59567	-0.78651			
24	2026-08-24	0.45433	-0.25671	-0.78651			
25	2026-08-24	0.45433	-0.25671	-0.82964			
26	2026-08-24	0.053665	-0.25671	-0.82964			
27	2026-08-24	0.053665	-0.21506	-0.82964			
28	2026-08-24	0.053665	-0.21506	-0.97769			
29	2026-08-24	-0.5005	-0.21506	-0.97769			
30	2026-08-24	-0.5005	-0.66821	-0.97769			
31	2026-08-24	-0.5005	-0.66821	-1.30305			
32	2026-08-24	-0.63846	-0.66821	-1.30305			
33	2026-08-24	-0.63846	-0.66821	-1.2177			
34	2026-08-24	-0.63846	-0.74335	-1.2177			
35	2026-08-24	0.009918	-0.74335	-1.2177			
36	2026-08-24	0.009918	-0.80145	-1.2177			
37	2026-08-24	0.009918	-0.80145	-0.13588			
38	2026-08-24	-0.01488	-0.80145	-0.13588			
39	2026-08-24	-0.01488	-0.80145	-0.89201			
40	2026-08-24	-0.01488	-0.83235	-0.89201			
41	2026-08-24	0.072128	-0.83235	-0.89201			
42	2026-08-24	0.072128	-0.19212	-0.89201			
43	2026-08-24	0.072128	-0.19212	-0.97588			
44	2026-08-24	0.072647	-0.19212	-0.97588			
45	2026-08-24	0.072647	-0.18352	-0.97588			
46	2026-08-24	0.072647	-0.18352	-0.98701			
47	2026-08-24	0.309433	-0.18352	-0.98701			
48	2026-08-24	0.309433	-0.34106	-0.98701			

< > accelerometer_combined +

